

DSCB140
Datenbanken und Datenkunde I

VL5 – Structured Query Language (SQL)
Teil 2: DML

Agenda

- **DML: Einfügen von Daten**
- ☐ **DML: Aktualisieren von Daten**
- ☐ **DML: Löschen von Daten**
- ☐ **Zusammenfassung und Ausblick**

Übung “Cloud SQL Setup”



- Übung (<https://pkuep.github.io/hska-140-db>)
 - Nun möchten wir die theoretischen Konzepte in einer „echten“ Umgebung umsetzen.
 - Insbesondere sollte die Datenbank auch strukturell bei allen Teilnehmern identisch sein (DDL-Teil). Dies werden wir zu Beginn der folgenden Übung sicherstellen.

DSCB140 - Datenbanken und Datenkunde

Einführung

[Herausforderungen dateibasierter Speicherung und erster Datenbankkontakt](#)

Normalisierung

[Übung Normalisierung](#)

SQL

Allgemein

[Setup der Übungsumgebung](#)

Data Definition Language

[Fallstudie Webshop - Tabellenerzeugung](#)

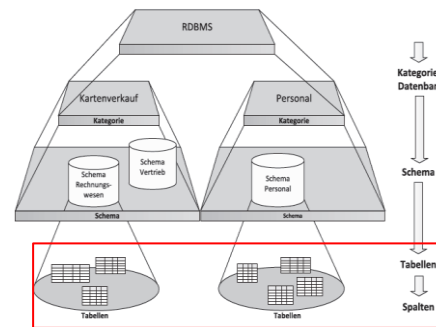
Data Manipulation Language

[Fallstudie Webshop - Befüllung der Tabellen \(inkl. DDL-Vorgabe\)](#)

Bis Schritt 4!



Einfügen von Daten



DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

- Daten werden mittels des INSERT-Statements eingefügt.
 - Wir beschränken uns auf die einfache Syntax zum Einfügen konkreter Daten (ohne Berechnungen, Partitionsdefinitionen etc.):

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
      [INTO] tbl_name
      [PARTITION (partition_name [, partition_name] ...)]
      [(col_name [, col_name] ...)]
      {VALUES | VALUE} (value_list) [, (value_list)] ...
```

value:

```
{expr | DEFAULT}
```

value_list:

```
value [, value] ...
```

Beispiel Einfügen von zwei Zeilen

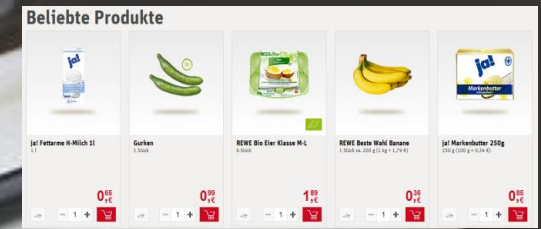
(1) Ohne Angabe von Spaltennamen
(Werte für alle Spalten müssen übergeben werden):

```
INSERT INTO status VALUES
(1, 'Offen'), (2, 'In Bearbeitung')
```

(2) Mit Angabe von Spaltennamen
(Werte für spezifizierte Spalten müssen übergeben werden):

```
INSERT INTO status (statusid) VALUES
(3), (4)
```

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/insert.html>

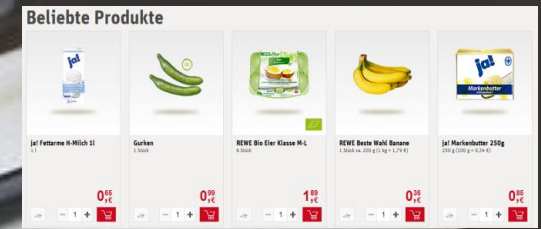


■ Vorbereitung:

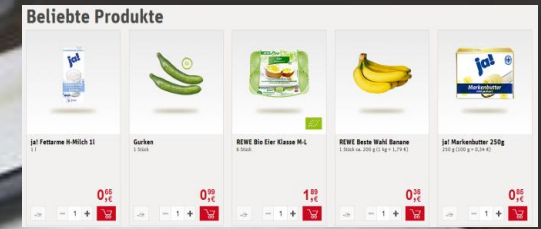
- Leeren Sie die Tabelle „abteilung“, „benutzer“ und „mitarbeiter“ mittels `DELETE FROM table_name;`

■ Einfügen von Abteilungen

- Fügen Sie eine Abteilung „Unternehmensleitung“ (abtid 1) und keiner übergeordneten Abteilung in der Hierarchie ein.
 - Variante 1: Nutzen Sie die Syntax zum übergeben von Werte für alle Spalten
 - Leeren Sie die Tabelle erneut.
 - Variante 2: Nutzen Sie die Syntax zum übergeben von Werte für einzelne Spalten
- Fügen Sie in einem Befehl
 - Abteilung „Einkauf“ (abtid 2) und „Unternehmensleitung“ als übergeordnete Abteilung,
 - Abteilung „Operativer Einkauf “ (abtid 3) und „Einkauf“ als übergeordnete Abteilung,
 - Abteilung „Strategischer Einkauf “ (abtid 4) und „Einkauf“ als übergeordnete Abteilung ein.



- Einfügen von Benutzer und Mitarbeiter
 - Fügen Sie einen Mitarbeiter „Herbert Meier“ ein (Mitarbeiternummer 123).
 - Sein Kennwort lautet „geheim“.
 - Die Emailadresse ist „meier@ab.de“.
 - Herr Meier hat die Telefonnummer „0721 123456“ und arbeitet in der „Unternehmensleitung“.
 - In welche Tabellen und in welcher Reihenfolge sind die Werte einzufügen?



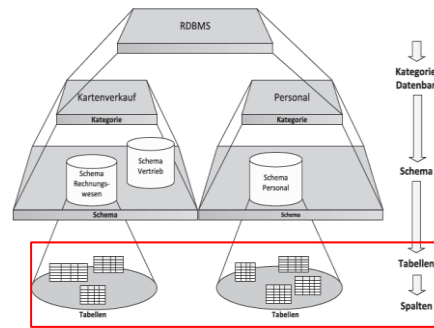
■ Schemaänderung

- Wir möchten nun (temporär) die min-Kardinalität der Mitarbeiter-Abteilung-Beziehung auf 0 ändern (ein Mitarbeiter muss also keiner Abteilung zugeordnet sein).
- Da wir nun Daten in der Datenbank haben benötigen wir nun **ALTER TABLE**.
- Schauen Sie in die Dokumentation (<https://dev.mysql.com/doc/refman/5.7/en/alter-table.html>) und setzen Sie die „Entfernung des NOT NULL-Constraints“ an entsprechender Stelle um.
 - Hinweis: Sie müssen MODIFY nutzen, um die „column_definition“ für die Spalte abtid neu zu setzen (ohne NOT NULL).
- Testen Sie, ob das Einfügen von NULL-Werten für abtid klappt.
 - Leer Sie dazu zunächst die Tabelle und fügen Sie eine neue Zeile für Herbert Meier (s.o.) ohne Abteilung ein.
- Versuchen Sie, die Änderung mittels ALTER TABLE rückgängig zu machen.
 - Es sollen also wieder Abteilungen zwingend angegeben werden.
 - Was fällt Ihnen auf?

Agenda

- ☐ **DML: Einfügen von Daten**
- ☒ **DML: Aktualisieren von Daten**
- ☐ **DML: Löschen von Daten**
- ☐ **Zusammenfassung und Ausblick**

Aktualisieren von Daten



DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

- Existierende Daten werden mittels des UPDATE-Statements verändert.
 - Wenn nicht alle Zeilen zu aktualisieren sind, dann ist mittels WHERE-Einschränkung zu definieren, welche Zeilen betroffen sind.
 - Die WHERE-Bedingungssyntax lernen wir im Detail im Bereich DQL.
 - Wird „DEFAULT“ anstelle eines konkreten Werts angegeben, dann wird der mittels DDL definierte Standardwert für die Spalte verwendet (haben wir bisher nicht genutzt)

```
UPDATE [LOW_PRIORITY] [IGNORE] table_reference
  SET assignment_list
  [WHERE where_condition]
  [ORDER BY ...]
  [LIMIT row_count]
```

```
value:
  {expr | DEFAULT}
```

```
assignment:
  col_name = value
```

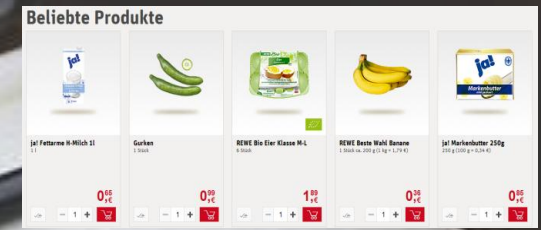
```
assignment_list:
  assignment [, assignment] ...
```

Beispiel: Aktualisierung mit Einschränkung
(Setzen der Statusbezeichnung für den Status mit ID 3):

```
UPDATE status
SET bearbeitungsstatus='Ausgeliefert'
WHERE statusid=3
```

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/update.html>

Fallstudie



■ Aktualisieren von Abteilungen

- Aktualisieren Sie die Abteilung „Operativer Einkauf“:
Nennen Sie die Abteilung „Disposition“.

- Sie benötigen hierzu das WHERE-statement (abtid=3).
 - UPDATE ... WHERE abtid = 3
- Die Formulierung von WHERE-statements werden wir in der DQL im Detail lernen.
- Die Syntax ist sehr mächtig, weshalb die Dokumentation relativ kompliziert wirkt:
<https://dev.mysql.com/doc/refman/5.7/en/expressions.html>

■ Aktualisierung von Mitarbeiter

- Weisen Sie „Herbert Meier“ (s.o.) wieder die Abteilung „Geschäftsleitung“ zu.
- Setzen Sie nun auf der Spalte abtid in Tabelle Mitarbeiter wieder das NOT NULL-Constraint.

```
expr:
  expr OR expr
  expr || expr
  expr XOR expr
  expr AND expr
  expr && expr
  NOT expr
  |
  | expr
  | boolean_primary IS [NOT] {TRUE | FALSE | UNKNOWN}
  | boolean_primary

boolean_primary:
  boolean_primary IS [NOT] NULL
  boolean_primary <=> predicate
  | boolean_primary comparison_operator predicate
  | boolean_primary comparison_operator {ALL | ANY} (subquery)
  | predicate

comparison_operator:
  = | > | < | <= | >= | <> | !=

predicate:
  bit_expr [NOT] IN (subquery)
  | bit_expr [NOT] IN (expr [, expr] ...)
  | bit_expr [NOT] BETWEEN bit_expr AND predicate
  | bit_expr SOUNDS LIKE bit_expr
  | bit_expr [NOT] LIKE simple_expr [ESCAPE simple_expr]
  | bit_expr [NOT] REGEXP bit_expr
  | bit_expr

bit_expr:
  bit_expr | bit_expr
  | bit_expr & bit_expr
  | bit_expr << bit_expr
  | bit_expr >> bit_expr
  | bit_expr + bit_expr
  | bit_expr - bit_expr
  | bit_expr * bit_expr
  | bit_expr / bit_expr
  | bit_expr DIV bit_expr
  | bit_expr MOD bit_expr
  | bit_expr % bit_expr
  | bit_expr ^ bit_expr
  | bit_expr + interval expr
  | bit_expr - interval expr
  | simple_expr

simple_expr:
  literal
  | identifier
  | function_call
  | simple_expr COLLATE collation_name
  | param_marker
  | variable
  | simple_expr || simple_expr
  | + simple_expr
  | - simple_expr
  | ~ simple_expr
  | ! simple_expr
  | BINARY simple_expr
  | (expr [, expr] ...)
  | ROW (expr, expr [, expr] ...)
  | (subquery)
  | EXISTS (subquery)
  | {identifier expr}
  | match_expr
  | case_expr
  | interval_expr
```

expr (=expression): **abtid = 3**

Unsere „expression“
(=logischer Ausdruck) besteht
nur aus einem
boolean_primary (also keine
UND/ODER-Verknüpfung).
boolean_primary: **abtid = 3**

Wir vergleichen zwei
predicates auf Gleichheit:
predicate1: **abtid**
comparison_operator: **=**
predicate2: **3**

Unsere beiden *predicates* sind
jeweils *bit_expr* und darin
simple_expr (einfache
Ausdrücke, in denen wir keine
Berechnungen vornehmen).

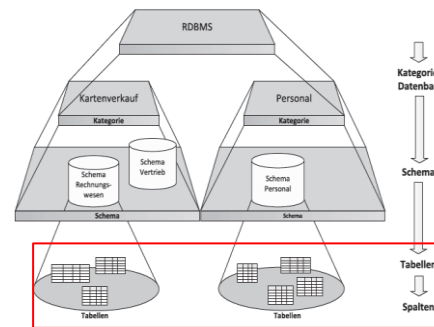
Die spezifizierte Zahl „3“ nennt
sich *literal* (im Speziellen ein
numerical literal).

Die Bezeichnung der Spalte
„abtid“ nennt sich *identifier*.

Agenda

- ☐ **DML: Einfügen von Daten**
- ☐ **DML: Aktualisieren von Daten**
- ☒ **DML: Löschen von Daten**
- ☐ **Zusammenfassung und Ausblick**

Löschen von Daten



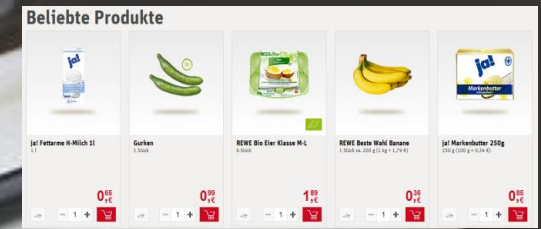
DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

- Existierende Daten werden mittels des DELETE-Statements gelöscht.
 - Wenn nicht alle Zeilen zu löschen sind, dann ist mittels WHERE-Einschränkung zu definieren, welche Zeilen betroffen sind.
 - Die WHERE-Bedingungssyntax lernen wir im Detail im Bereich DQL.

```
DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name
    [PARTITION (partition_name [, partition_name] ...)]
    [WHERE where_condition]
    [ORDER BY ...]
    [LIMIT row_count]
```

Beispiel: Löschen mit Einschränkung
(Löschen aller Zustände mit ID >= 3):

```
DELETE FROM status
WHERE statusid >= 3
```



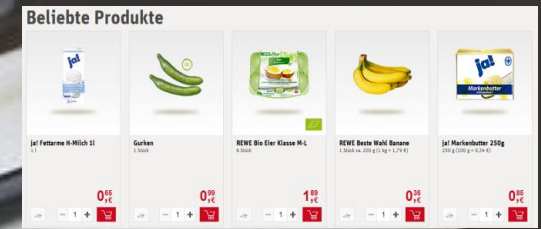
■ Löschen von Abteilungen

■ Löschen Sie die Abteilung „Strategischer Einkauf“

- Variante 1: Löschen Sie unter Angabe der abtid.
 - Sie benötigen hierzu das WHERE-statement (abtid=1).
 - Fügen Sie danach die Abteilung wieder ein (s.o.).
- Variante 2: Löschen Sie unter Angabe der Abteilungsbezeichnung.
 - Sie benötigen hierzu das WHERE-statement (bezeichnung="Strategischer Einkauf").
 - Es soll Groß- und Kleinschreibung ignoriert werden, d.h. auch wenn z.B. "StrateGIScher Einkauf" geschrieben wurde, soll dieser Datensatz gelöscht werden.
 - » Überlegen Sie sich, wie man dies umsetzen könnte.

■ Löschen Sie alle Abteilungen, die „Einkauf“ in ihrer Bezeichnung enthalten.

- „Einkauf“ soll an beliebiger Stelle auftauchen können (Groß-/Kleinschreibung irrelevant).
- Sie benötigen auch hier ein WHERE-statement.
- Schauen Sie sich den LIKE-Operator dazu an:
<https://dev.mysql.com/doc/refman/5.7/en/pattern-matching.html>



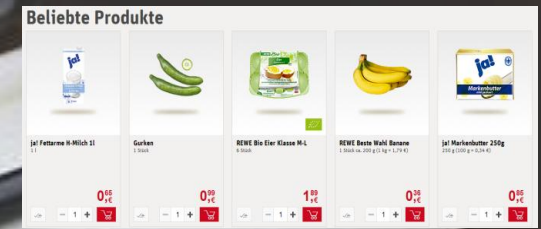
■ Löschen von Abteilungen

■ Löschen Sie die Abteilung „Strategischer Einkauf“

- Variante 1: Löschen Sie unter Angabe der abtid.
 - Sie benötigen hierzu das WHERE-statement (abtid=1).
 - Fügen Sie danach die Abteilung wieder ein (s.o.).
- Variante 2: Löschen Sie unter Angabe der Abteilungsbezeichnung.
 - Sie benötigen hierzu das WHERE-statement (bezeichnung="Strategischer Einkauf").
 - Es soll Groß- und Kleinschreibung ignoriert werden, d.h. auch wenn z.B. "StrateGIScher Einkauf" geschrieben wurde, soll dieser Datensatz gelöscht werden.
 - » Überlegen Sie sich, wie man dies umsetzen könnte.

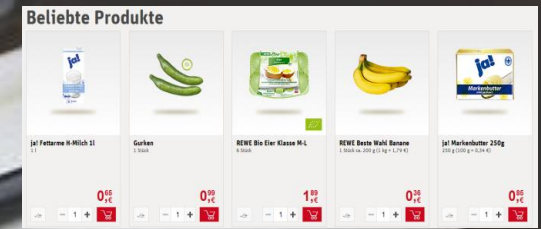
■ Löschen Sie alle Abteilungen, die „Einkauf“ in ihrer Bezeichnung enthalten.

- „Einkauf“ soll an beliebiger Stelle auftauchen können (Groß-/Kleinschreibung irrelevant).
- Sie benötigen auch hier ein WHERE-statement.
- Schauen Sie sich den LIKE-Operator dazu an:
<https://dev.mysql.com/doc/refman/5.7/en/pattern-matching.html>



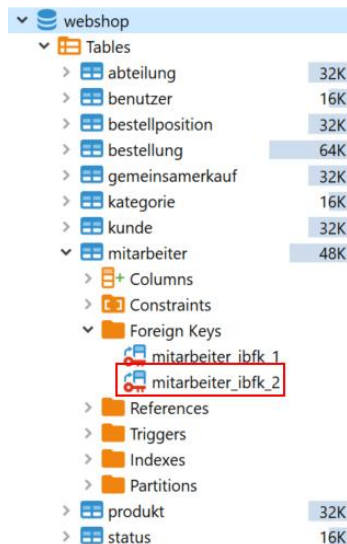
■ Löschen von Abteilungen

- Schauen Sie sich die Tabellen Mitarbeiter und Abteilung an (select * from table_name).
- Löschen Sie die Abteilung „Geschäftsleitung“ und betrachten Sie danach die Tabellen erneut.
 - Was ist passiert?
 - Warum wurden weitere Abteilungen gelöscht?
 - Warum wurde der Benutzer nicht gelöscht?
- An welcher Stelle wird die entsprechende Einstellung gemacht?
- Schauen Sie dazu in die DDL-Definition, die wir oben aus Python eingespielt haben:
[Link zur SQL DDL-Definition](#)

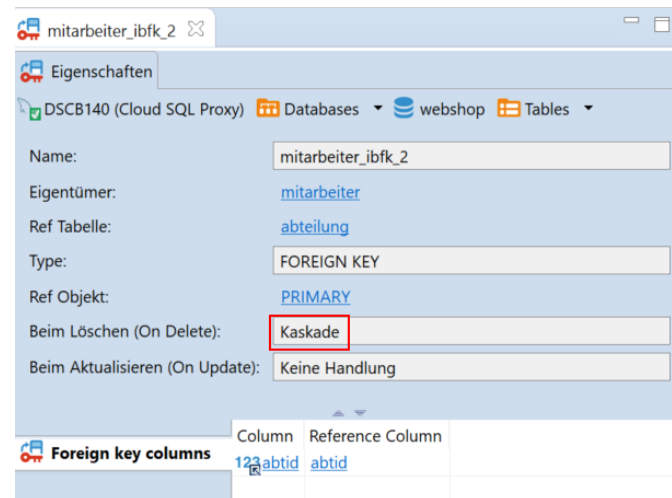


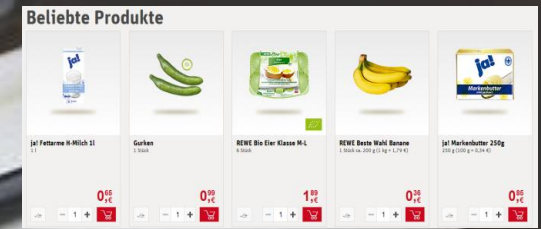
■ Anpassung der Löschweitergabe

- Wir möchten nun (temporär) die Löschweitergabe entfernen und andere Einstellungen für Fremdschlüssel ausprobieren (am Beispiel der abtid).
- Dazu müssen wir zunächst den MySQL-internen Namen des Fremdschlüssels ermitteln.
 - Die Definition einer Tabelle (inkl. Fremdschlüsselnamen) können Sie sich mit „show create table table_name“ anzeigen lassen. Führen Sie dies für „mitarbeiter“ durch.
 - Alternativ können Sie dies auch in DBeaver in der Hierarchie anzeigen:



- Doppelklick auf den Fremdschlüssel zum Anzeigen der Definition: (dort sehen Sie die „kaskadierende“ Löschweitergabe





- Anpassung der Löschweitergabe (cont'd)
 - Für die Anpassung der Löschweitergabe nutzen wir ALTER TABLE.
 - <https://dev.mysql.com/doc/refman/5.7/en/alter-table.html>
 - Schauen Sie sich zuerst die Optionen für „on delete“ bei Fremdschlüsseln an:
 - <https://dev.mysql.com/doc/refman/5.7/en/create-table.html>
 - Sie müssen als erstes den Fremdschlüssel mittels ALTER TABLE löschen.
 - Danach müssen Sie den Fremdschlüssel neu definieren.
 - Dieses Mal mit der Einstellung „on delete restrict“ am Ende der Fremdschlüsseldefinition.
 - Fügen Sie danach folgende Abteilung und Mitarbeiter ein:
 - insert into abteilung (abtid, bezeichnung) values (1, 'Unternehmensleitung');
 - insert into mitarbeiter values ('meier@ab.de', 123, '0721 123456', 1);
 - Versuchen Sie nun die Abteilung zu löschen.
 - Machen Sie die Einstellung rückgängig, d.h. der Fremdschlüssel soll wieder mit „on delete cascade“ definiert sein. Löschen Sie im Anschluss erneut die Abteilung.

Übung “Cloud SQL Setup”



- Übung (<https://pkuep.github.io/hska-140-db>)
 - Nun möchten wir die theoretischen Konzepte in einer „echten“ Umgebung umsetzen.
 - Insbesondere sollte die Datenbank auch strukturell bei allen Teilnehmern identisch sein (DDL-Teil). Dies werden wir zu Beginn der folgenden Übung sicherstellen.

DSCB140 - Datenbanken und Datenkunde

Einführung

[Herausforderungen dateibasierter Speicherung und erster Datenbankkontakt](#)

Normalisierung

[Übung Normalisierung](#)

SQL

Allgemein

[Setup der Übungsumgebung](#)

Data Definition Language

[Fallstudie Webshop - Tabellenerzeugung](#)

Data Manipulation Language

[Fallstudie Webshop - Befüllung der Tabellen \(inkl. DDL-Vorgabe\)](#)

Ab Schritt 6!



Agenda

- ☐ **DML: Einfügen von Daten**
- ☐ **DML: Aktualisieren von Daten**
- ☐ **DML: Löschen von Daten**
- ☒ **Zusammenfassung und Ausblick**



- In der letzten Vorlesung haben wir uns mit der Data Manipulation Language anhand von MySQL auseinandergesetzt.
 - Dabei haben wir nicht nur mittels DBeaver (also einem Datenbank-Client für die Entwicklung) Daten manipuliert, sondern auch gesehen, wie aus Python mit einem RDBMS in einem Basisansatz gearbeitet werden kann.
- Sie sollten nun
 - Tabellen befüllen können sowie
 - existierende Daten einzeln sowie nach bestimmten Bedingungen manipulieren und löschen können.
- Darüber hinaus haben Sie den Umgang mit ALTER TABLE vereinzelt kennengelernt.
- In den folgenden Veranstaltungen werden wir „Data Science“ mit SQL betreiben, d.h. Daten einzeln sowie aggregiert abrufen und aufbereiten. Dazu nutzen wir die Data Query Language von SQL.
 - Dabei gehen wir auch noch intensiv auf WHERE-Bedingungen ein, die auch in der DML eingesetzt werden.

Vielen Dank für Ihre Aufmerksamkeit!

